

第13章 数据处理与分析

人工智能学院
古 晶

西安电子科技大学



第13章 数据存储与管理



- 13.1 数据处理与分析的概念
- 13.2 机器学习和数据挖掘算法
- 13.3 大数据处理与分析技术
- 13.4 大数据处理与分析代表性产品
- 13.5 知识点小结



13.1 数据处理与分析的概念

- 数据分析可以分为**广义**的数据分析和**狭义**的数据分析，**广义的数据分析就包括狭义的数据分析和数据挖掘。**
- **广义的数据分析**是指用适当的分析方法（来自统计学、机器学习和数据挖掘等领域），对收集来的数据进行分析，提取有用信息和形成结论的过程。
- 可以看出，在**广义的数据分析**中，可以使用**复杂的机器学习和数据挖掘算法**，也可以只使用一些简单的统计分析方法，比如汇总求和、求平均值、求均方差等。
- **狭义的数据分析**是指根据分析目的，用适当的**统计分析方法**及工具，对收集来的数据进行处理与分析，提取有价值的信息。



13.1.1 数据分析与数据挖掘

狭义的数据分析和数据挖掘是有着明显的区分的，具体如下：

- 在**定义层面**。**狭义的数据分析**在上面已经定义。而**数据挖掘**是指从大量的数据中，通过统计学、人工智能、机器学习等方法，**挖掘出未知的、且有价值的信息和知识的过程**。
- 在**作用层面**。**数据分析**主要实现三大作用：**现状分析、原因分析、预测分析（定量）**。数据分析的目标明确，先做假设，然后通过数据分析来验证假设是否正确，从而得到相应的结论。**数据挖掘**主要侧重解决四类问题：**分类、聚类、关联和预测**（定量、定性），数据挖掘的重点在寻找未知的模式与规律；如著名的数据挖掘案例——啤酒与尿布，就是事先未知的，但又是非常有价值的信息。



13.1.1 数据分析与数据挖掘

- 在**方法层面**。**数据分析**主要采用**对比分析**、**回归分析**等常用分析方法；**数据挖掘**主要采用**决策树**、**神经网络**等统计学、**机器学习**等方法进行挖掘；
- 在**结果层面**。**数据分析**一般都是得到一个**指标统计量结果**，如总和、平均值等，这些指标数据都需要与业务结合进行解读，才能发挥出数据的价值与作用。**数据挖掘**则是**输出模型或规则**，并且可相应得到模型得分或标签，模型得分如流失概率值、总和得分、相似度、预测值等，标签如高中低价值用户、流失与非流失、信用优良中差等。

综上所述，**数据分析（狭义）**与**数据挖掘**的**本质**都是一**样**的，都是从数据里面发现关于业务的知识（有价值的信息），从而帮助业务运营、改进产品以及帮助企业做更好的决策。所以，数据分析（狭义）与数据挖掘构成广义的数据分析。



13.1.2 数据分析与数据处理

- **数据分析过程通常会伴随着发生数据处理**（或者说伴随着大量数据计算），**数据分析**和**数据处理**是一对关系紧密的概念。
- 也就是说，当用户在进行数据分析的时候，底层的计算机系统会根据数据分析任务的要求，**使用程序进行大量的数据处理**（或者说发生大量的数据计算）。
- **例如**，当用户进行决策树分析时，需要事先根据决策树算法编写分析程序，当分析开始以后，决策树分析程序就会从磁盘读取数据进行大量计算，最终给出计算结果（也就是决策树分析结果）。



13.1.3 大数据处理与分析

- 数据分析包含两个要素，即**理论**和**技术**。
- 在**理论层面**，需要统计学、机器学习和数据挖掘等知识；
- 在**技术层面**，包括单机**分析工具**（比如SPSS、SAS等）或单机**编程语言**（比如Python、R），以及**大数据处理与分析技术**（比如MapReduce、Spark、Hive等）。
- 数据分析可以是对小规模数据的分析，也可是对大规模数据的分析（即“**大数据分析**”）。
- 在**大数据时代到来之前**，**数据分析**主要以**小规模的数据**为主，一般使用统计学、机器学习和数据挖掘的相关方法，以单机分析工具（比如SPSS）或者单机编程（比如Python）的方式来实现分析程序。
- 到了**大数据时代**，数据量爆炸式地增长，很多时候需要对规模巨大的全量数据进行分析，这时，单机工具和单机程序已经显得“无能为力”，就需要采用**分布式**实现技术，比如使用MapReduce编写分布式分析程序，借助于集群的多台机器进行并行数据处理分析，这个过程就被称为“**大数据处理与分析**”。



第13章 数据存储与管理

- 13.1 数据处理与分析的概念
- 13.2 机器学习和数据挖掘算法**
- 13.3 大数据处理与分析技术
- 13.4 大数据处理与分析代表性产品
- 13.5 知识点小结



13.2.1 机器学习和数据挖掘算法概述

机器学习是一门多领域交叉学科，涉及概率论、统计学、逼近论、凸分析、算法复杂度理论等多门学科，**专门研究计算机怎样模拟或实现人类的学习行为**，以获取新的知识或技能，重新组织已有的知识结构使之不断改善自身的性能，它是人工智能的核心，是使计算机具有智能的根本途径，其应用遍及人工智能的各个领域。

数据挖掘是指**从大量的数据中通过算法搜索隐藏于其中信息的过程**。数据挖掘可以视为机器学习与数据库的交叉，它主要利用机器学习界提供的算法来分析海量数据，利用数据库界提供的存储技术来管理海量数据。



13.1.1 机器学习和数据挖掘算法概述

- 虽然**数据挖掘**的很多技术都来自**机器学习**领域，但是，我们并不能因此就认为数据挖掘只是机器学习的简单应用。毕竟，机器学习通常只研究小规模的数据对象，往往无法应用到海量数据的情形，**数据挖掘领域必须借助于海量数据管理技术对数据进行存储和处理**，同时对一些传统的机器学习算法进行改进，使其能够支持海量数据的情形。
- **典型的机器学习和数据挖掘算法**包括**分类、聚类、回归分析和关联规则**等。



13.2.2 分类



- **分类**是一种重要的机器学习和数据挖掘技术。分类的**目的**是根据数据集的特点构造一个分类函数或分类模型(也常常称作分类器), 该模型能把未知类别的样本映射到给定类别中。
- 构造分类模型的过程一般分为**训练**和**测试**两个阶段。在构造模型之前, 将数据集随机地分为训练数据集和测试数据集。先使用训练数据集来构造分类模型, 然后使用测试数据集来评估模型的分类准确率。如果认为模型的准确率可以接受, 就可以用该模型对其它数据元组进行分类。一般来说, 测试阶段的代价远低于训练阶段。
- 典型的分类方法包括**决策树**、**朴素贝叶斯**、**支持向量机**和**人工神经网络**等。



13.2.2 分类



- 这里给出一个分类的**应用实例**。
- 假设有一名植物学爱好者对她发现的鸢尾花的品种很感兴趣。她收集了每朵鸢尾花的一些测量数据：花瓣的长度和宽度以及花萼的长度和宽度。她还有一些鸢尾花分类的数据，也就是说，这些花之前已经被植物学专家鉴定为属于setosa、versicolor或virginica三个品种之一。
- 基于这些分类数据，她可以确定每朵鸢尾花所属的品种。于是，她可以构建一个**分类算法**，让算法从这些已知品种的鸢尾花测量数据中进行学习得到**分类模型**，再使用分类模型预测新鸢尾花的品种。



13.2.3 聚类



聚类分析的**常见应用场景**包括：

(1) **目标用户的群体分类**。通过对特定运营目的和商业目的所挑选出的指标变量进行聚类分析，把目标群体划分成几个具有明显特征区别的细分群体，从而可以在运营活动中为这些细分群体采取精细化，个性化的运营和服务，最终提升运营的效率和商业效果。

(2) **探测发现离群点和异常值**。这里的**离群点**是指相对于整体数据对象而言的**少数数据对象**，这些对象的行为特征与整体的数据行为特征很不一致，比如，某B2C电商平台上，比较昂贵、频繁的交易，就有可能隐含欺诈的风险，需要风控部门提前关注。



13.2.3 聚类

聚类又称**群分析**，是一种重要的机器学习和数据挖掘技术。

聚类分析的**目的**是将数据集中的数据对象划分到若干个簇中，并且保证每个簇之间样本尽量接近，不同簇的样本间距离尽量远。通过聚类生成的簇是一组数据对象的集合，簇满足以下**两个条件**：

- **每个簇至少包含一个数据对象；**
- **每个数据对象仅属于一个簇。**



13.2.3 聚类



- 聚类分析一般属于**无监督分类**的范畴，按照一定的要求和规律，**在没有关于分类的先验知识情况下，对数据进行区分和分类。**
- 聚类既能作为一个单独过程，用于找寻数据内部的分布结构，也可以作为分类等其他学习任务的前驱过程。
- **聚类算法**可分为**划分法**、**层次法**、基于**密度**的方法、基于**网格**的方法、基于**模型**的方法。
- 这些方法没有统一的评价指标，因为不同聚类算法的目标函数相差很大。有些聚类是**基于距离**的（如K-Means），有些是**假设先验分布**的（如GMM，LDA），有些是带有**图聚类**和**谱分析**性质的（如谱聚类），还有些是**基于密度**的（如DBSCAN）。**聚类算法应该嵌入到问题中进行评价。**



13.2.4 回归分析

- **回归分析** (Regression Analysis)指的是确定两种或两种以上变量间相互依赖的定量关系的一种统计分析方法。
- 回归分析按照**涉及的变量**的多少, 分为**一元回归**和**多元回归**分析;
- 按照**因变量**的多少, 可分为**简单回归**分析和**多重回归**分析;
- 按照**自变量和因变量之间的关系类型**, 可分为**线性回归**分析和**非线性回归**分析。
- 在大数据分析中, 回归分析是一种预测性的建模技术, 它研究的是**因变量 (目标) 和自变量 (预测器) 之间的关系**。这种技术通常用于**预测分析**, 时间序列模型以及发现变量之间的因果关系。**例如**, 司机的鲁莽驾驶与道路交通事故数量之间的关系, 最好的研究方法就是回归。



13.2.5 关联规则



- 关联规则最初针对**购物篮分析**问题提出。
- 零售商想了解顾客可能在一次购物时同时购买哪些商品，可以对商店的顾客事物零售数量进行购物篮分析，即通过发现顾客放入“购物篮”中的不同商品之间的关联，分析顾客的购物习惯。这种关联可以帮助零售商了解哪些商品频繁地被顾客同时购买，从而辅助他们制定营销策略。
- 关联规则**定义**为：假设是 $I = \{I_1, I_2, I_3, \dots, I_m\}$ 项的集合。给定一个交易数据库 D ，其中每个事务 t 是 I 的非空子集，即每一个交易都与一个唯一的标识符 TID 对应。关联规则在 D 中的支持度是 D 中事务同时包含 X 、 Y 的百分比，即**概率**；置信度是 D 中事务已经包含 X 的情况下，包含 Y 的百分比，即**条件概率**。如果满足**最小支持度阈值**和**最小置信度阈值**，则认为**关联规则**是有趣的。这些阈值是根据挖掘需要人为设定的。



13.2.5 关联规则

- 这里**举例**说明。下表是顾客购买记录的数据库 D ，包含6个事务。
项集 $I = \{\text{乒乓球拍, 乒乓球, 运动鞋, 羽毛球}\}$ 。
- 考虑**关联规则**（频繁二项集）：乒乓球拍与乒乓球
- 事务1,2,3,4,6包含**乒乓球拍**，事务1,2,6同时包含**乒乓球拍和乒乓球**，这里用**X**表示购买了乒乓球，用**Y**表示购买了乒乓球拍，则 $X \wedge Y = 3$, $D = 6$ ，支持度 $(X \wedge Y) / D = 0.5$ ； $X = 5$ ，置信度 $(X \wedge Y) / X = 0.6$ 。
若给定最小支持度 $\alpha = 0.5$ ，最小置信度 $\beta = 0.6$ ，**认为购买乒乓球拍和购买乒乓球之间存在关联。**

顾客购买记录

TID	乒乓球拍	乒乓球	运动鞋	羽毛球
1	1	1	1	0
2	1	1	0	0
3	1	0	0	0
4	1	0	1	0
5	0	1	1	1
6	1	1	0	0

常见的**关联规则挖掘算法**包括**Apriori算法**和**FP-Growth算法**等。



13.2.6 协同过滤



推荐技术从被提出到现在已有十余年。**协同过滤**作为最早、最知名的推荐算法，不仅在学术界得到了深入研究，而且至今在业界仍有广泛的应用，已经被大量应用到电子商务的推荐系统中。协同过滤主要包括**基于用户**的协同过滤、**基于物品**的协同过滤和**基于模型**的协同过滤。

- **基于用户的协同过滤算法**（简称UserCF算法）是**推荐系统**中最古老的算法，直到现在该算法都是推荐系统领域最著名的算法之一。**UserCF算法**符合人们对于“**趣味相投**”的认知，即**兴趣相似的用户往往有相同的物品喜好**，当目标用户需要个性化推荐时，可以先找到和目标用户有相似兴趣的用户群体，然后将这个用户群体喜欢的、而目标用户没有听说过的物品推荐给目标用户，这种方法就称为“基于用户的协同过滤算法”。



13.2.6 协同过滤

- **基于物品的协同过滤算法**（简称**ItemCF算法**）是目前业界应用最多的算法。ItemCF算法并不利用物品的内容属性计算物品之间的相似度，而主要通过分析用户的行为记录来计算物品之间的相似度，该算法基于的假设是：**物品A和物品B具有很大的相似度是因为喜欢物品A的用户大多也喜欢物品B**。例如，该算法会因为你购买过《数据挖掘导论》而给你推荐《机器学习实战》，因为买过《数据挖掘导论》的用户多数也购买了《机器学习实战》。
- **基于模型的协同过滤算法**（**ModelCF**）是通过已经观察到的所有用户给产品的打分，来推断每个用户的喜好并向用户推荐适合的产品。实际上，ModelCF同时考虑了**用户和物品**两个方面，因此，**它也可以看作是UserCF和ItemCF的混合形式**。



第13章 数据存储与管理

- 13.1 数据处理与分析的概念
- 13.2 机器学习和数据挖掘算法
- 13.3 大数据处理与分析技术**
- 13.4 大数据处理与分析代表性产品
- 13.5 知识点小结



13.3.1 技术分类

大数据计算模式及其代表产品

大数据计算模式	解决问题	代表产品
批处理计算	针对大规模数据的批量处理	MapReduce、Spark等
流计算	针对流数据的实时计算	Flink、Storm、S4、Spark Streaming、Flume、Streams、Puma、DStream、Super Mario、银河流数据处理平台等
图计算	针对大规模图结构数据的处理	Pregel、GraphX、Giraph、PowerGraph、Hama、GoldenOrb等
查询分析计算	大规模数据的存储管理和查询分析	Dremel、Hive、Cassandra、Impala等

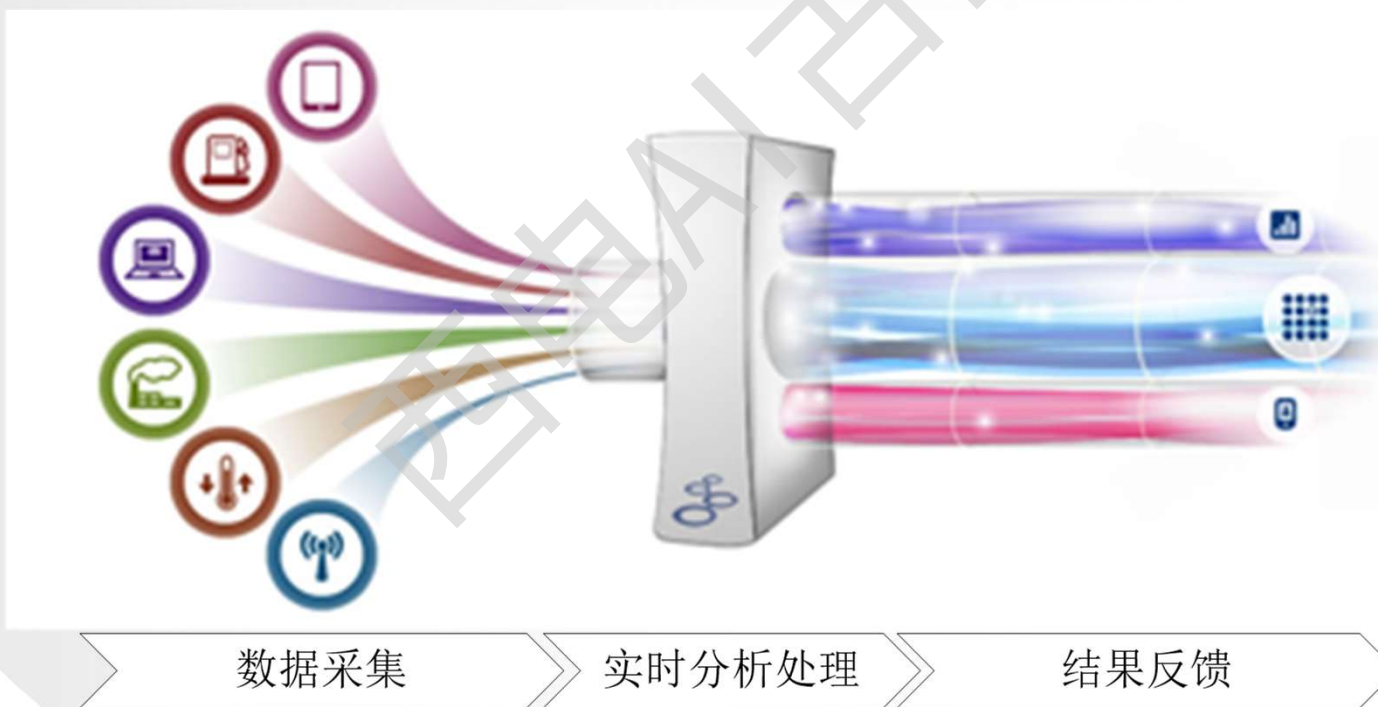


13.3.2 流计算



(1) 流计算概念

流计算：**实时获取**来自不同数据源的海量数据，经过**实时分析**处理，获得有价值的信息。



流计算示意图

西安电子科技大学



13.3.2 流计算



1) 流计算秉承一个基本理念, 即**数据的价值随着时间的流逝而降低**, 如用户点击流。因此, 当事件出现时就应该立即进行处理, 而不是缓存起来进行批量处理。为了及时处理流数据, 就需要一个低延迟、可扩展、高可靠的处理引擎。

2) 对于一个**流计算系统**来说, 它应达到如下需求:

- **高性能**: 处理大数据的基本要求, 如每秒处理几十万条数据。
- **海量式**: 支持TB级甚至是PB级的数据规模。
- **实时性**: 保证较低的延迟时间, 达到秒级别, 甚至是毫秒级别。
- **分布式**: 支持大数据的基本架构, 必须能够平滑扩展。
- **易用性**: 能够快速进行开发和部署。
- **可靠性**: 能可靠地处理流数据。

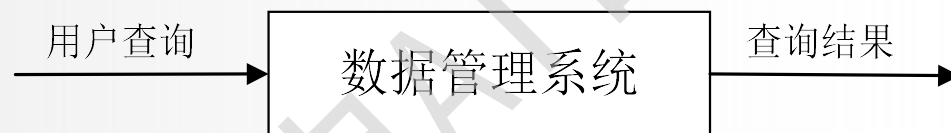


13.3.2 流计算



(2) 流计算的处理流程

传统的数据处理流程，需要先采集数据并存储在关系数据库等数据管理系统中，之后由用户通过查询操作和数据管理系统进行交互。



传统的数据处理流程示意图

传统的数据处理流程隐含了两个前提：

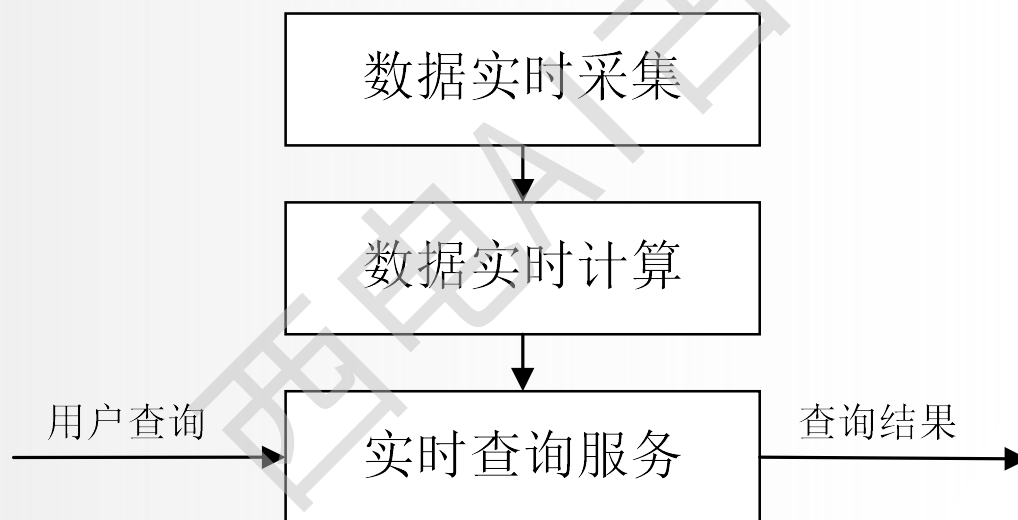
- **存储的数据是旧的。** 存储的静态数据是过去某一时刻的快照，这些数据在查询时可能已不具备时效性了。
- **需要用户主动发出查询来获取结果。**



13.3.2 流计算



流计算的处理流程一般包含三个阶段：**数据实时采集**、**数据实时计算**、**实时查询服务**。



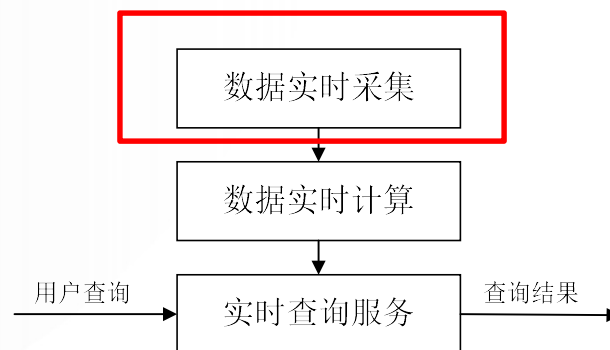
流计算处理流程示意图



13.3.2 流计算



- ❖ **数据实时采集阶段**通常采集多个数据源的海量数据，需要保证实时性、低延迟与稳定可靠。
- ❖ **以日志数据为例**，由于分布式集群的广泛应用，数据分散存储在不同的机器上，因此需要实时汇总来自不同机器上的日志数据。
- ❖ 目前有许多互联网公司发布的开源分布式日志采集系统均可满足每秒数百MB的数据采集和传输需求，如：
 - Facebook的Scribe
 - LinkedIn的Kafka
 - 淘宝的Time Tunnel
 - 基于Hadoop的Chukwa和Flume

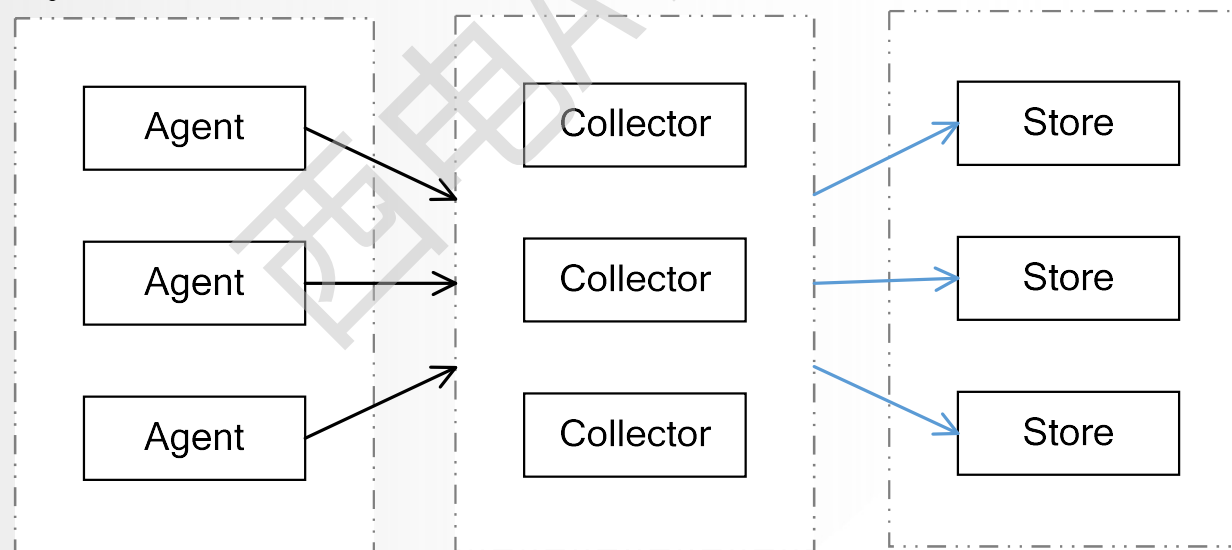




13.3.2 流计算



- ❖ 数据采集系统的基本架构一般有以下三个部分：
 - **Agent**: 主动采集数据，并把数据推送到Collector部分。
 - **Collector**: 接收多个Agent的数据，并实现有序、可靠、高性能的转发。
 - **Store**: 存储Collector转发过来的数据（对于流计算不存储数据）。



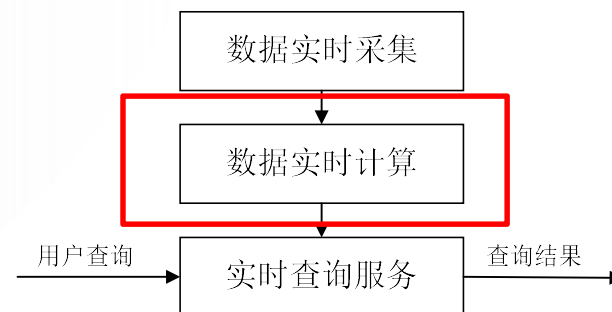
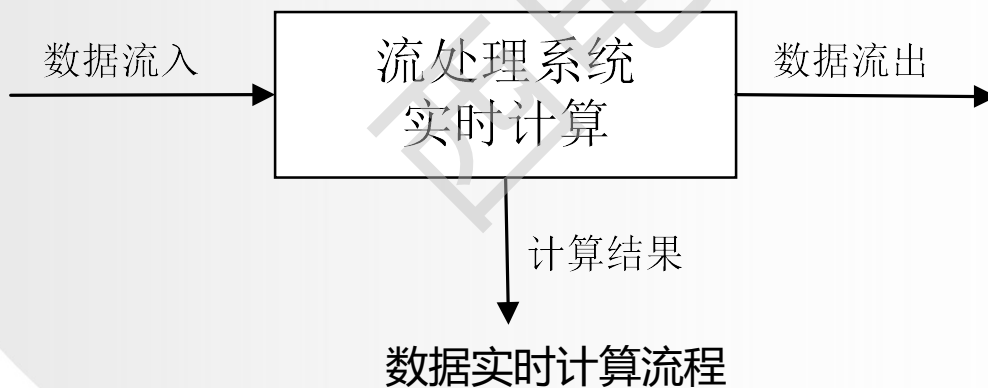
数据采集系统基本架构



13.3.2 流计算



- ❖ **数据实时计算阶段**对采集的数据进行实时的分析和计算，并反馈实时结果。
- ❖ 经流处理系统处理后的数据，可视情况进行存储，以便之后再进行分析计算。在时效性要求较高的场景中，处理之后的数据也可以直接丢弃。

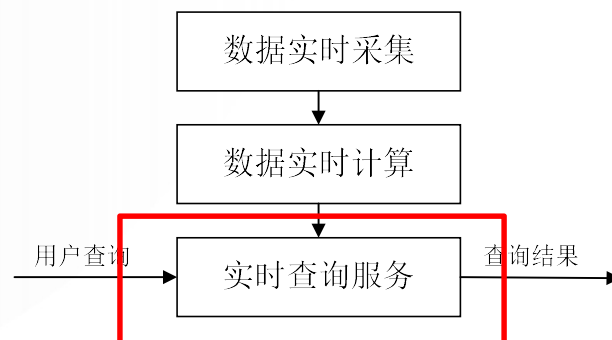




13.3.2 流计算



- ❖ **实时查询服务**：经由流计算框架得出的结果可供用户进行实时查询、展示或储存。
- ❖ **传统的数据处理流程**，用户需要主动发出查询才能获得想要的结果。而在**流处理流程**中，实时查询服务可以不断更新结果，并将用户所需的结果实时推送给用户。
- ❖ 虽然通过对传统的数据处理系统进行**定时**查询，也可以实现不断地更新结果和结果推送，但通过这样的方式获取的结果，仍然是根据过去某一时刻的数据得到的结果，与实时结果有着本质的区别。





13.3.2 流计算



流处理系统与传统的数据处理系统有如下不同：

- 流处理系统处理的是**实时**的数据，而传统的数据处理系统处理的是预先存储好的**静态**数据。
- 用户通过流处理系统获取的是**实时**结果，而通过传统的~~数据处理系统~~，获取的是**过去某一时刻**的结果。
- 流处理系统无需用户主动发出查询，实时查询服务可以主动将实时结果推送给用户。



13.3.3 图计算

传统图计算解决方案的**不足之处**:

- 常常表现出比较差的内存访问局部性。
- 针对单个顶点的处理工作过少。
- 计算过程中伴随着并行度的改变。



13.3.3 图计算



针对大型图（比如社交网络和网络图）的计算问题，**可能的解决方案及其不足之处**具体如下：

- 1) 为特定的图应用定制相应的分布式实现**：通用性不好。
- 2) 基于现有的分布式计算平台进行图计算**：在性能和易用性方面往往无法达到最优：
 - 现有的并行计算框架像MapReduce还无法满足复杂的关联性计算。
 - MapReduce作为单输入、两阶段、粗粒度数据并行的分布式计算框架，在表达多迭代、稀疏结构和细粒度数据时，力不从心。
 - 比如，有公司利用MapReduce进行社交用户推荐，对于5000万注册用户，50亿关系对，利用10台机器的集群，需要超过10个小时的计算。
- 3) 使用单机的图算法库**：比如BGL、LEAD、NetworkX、JDSL、Stanford GraphBase和FGL等，但是，在可以解决的问题的规模方面具有很大的局限性。
- 4) 使用已有的并行图计算系统**：比如，Parallel BGL和CGM Graph，实现了很多并行图算法，但是，对大规模分布式系统非常重要的一些方面（比如容错），无法提供较好的支持。



13.3.3 图计算



(2) 图计算通用软件

- ❖ 传统的图计算解决方案无法解决大型图的计算问题，因此，就需要设计能够用来解决这些问题的通用图计算软件。
- ❖ 针对大型图的计算，目前通用的图计算软件主要包括两种：
 - 第一种主要是**基于遍历算法的、实时的图数据库**，如Neo4j、OrientDB、DEX和 Infinite Graph。
 - 第二种则是**以图顶点为中心的、基于消息传递批处理的并行引擎**，如GoldenOrb、Giraph、Pregel和Hama，这些图处理软件主要是**基于BSP模型实现的并行图处理系统**。

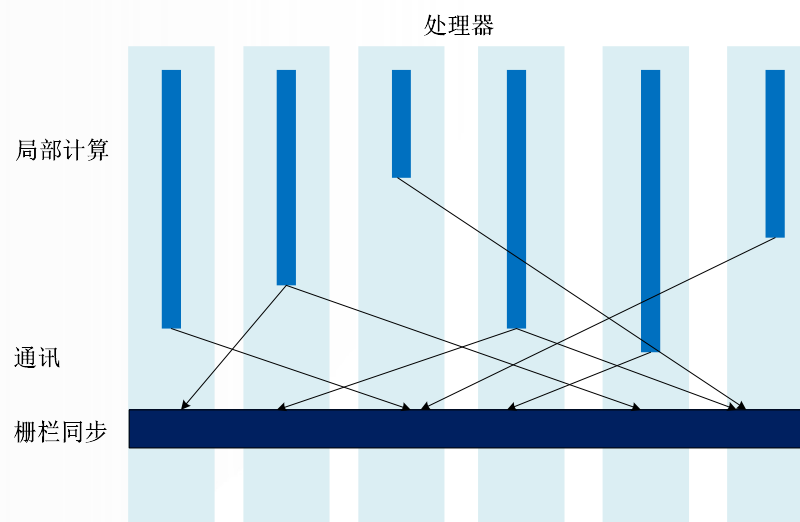
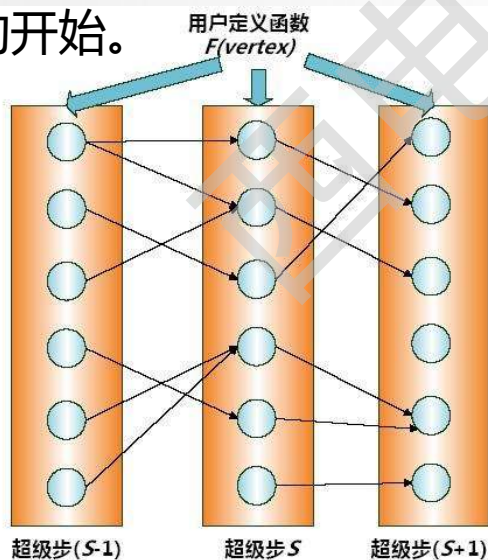


13.3.3 图计算



一次BSP(Bulk Synchronous Parallel Computing Model, 又称“大同步”模型)计算过程包括一系列全局超步(所谓的**超步就是计算中的一次迭代**), 每个超步主要包括三个组件:

- **局部计算**: 每个参与的**处理器**都有自身的计算任务, 它们只读取存储在本地内存中的值, 不同处理器的计算任务都是异步并且独立的。
- **通讯**: 处理器群相互交换数据, 交换的形式是, 由一方发起推送(put)和获取(get)操作。
- **栅栏同步**(Barrier Synchronization): 当一个处理器遇到“路障”(或栅栏), 会等到其他所有处理器完成它们的计算步骤; 每一次同步也是一个超步的完成和下一个超步的开始。



一个超步的垂直结构图



第13章 数据存储与管理

- 13.1 数据处理与分析的概念
- 13.2 机器学习和数据挖掘算法
- 13.3 大数据处理与分析技术
- 13.4 大数据处理与分析代表性产品**
- 13.5 知识点小结



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.1 分布式计算框架MapReduce

(1) MapReduce简介

- MapReduce是谷歌公司的核心计算模型。MapReduce将复杂的、运行于大规模集群上的并行计算过程高度地抽象到两个函数：**Map**和**Reduce**，这两个函数及其核心思想都源自**函数式编程语言**。
- 在MapReduce中，一个存储在分布式文件系统的大规模数据集会被切分成许多独立的小数据块，这些小数据块可以被多个Map任务并行处理。
- MapReduce框架会为每个**Map**任务输入一个数据子集，**Map**任务生成的结果会继续作为Reduce任务的输入，最终由**Reduce**任务输出最后结果，并写入分布式文件系统。
- 特别需要注意的是，适合用MapReduce来处理的数据集需要满足一个**前提条件**：**待处理的数据集可以分解成许多小的数据集，而且每一个小数据集都可以完全并行地进行处理。**



13.4.1 分布式计算框架MapReduce

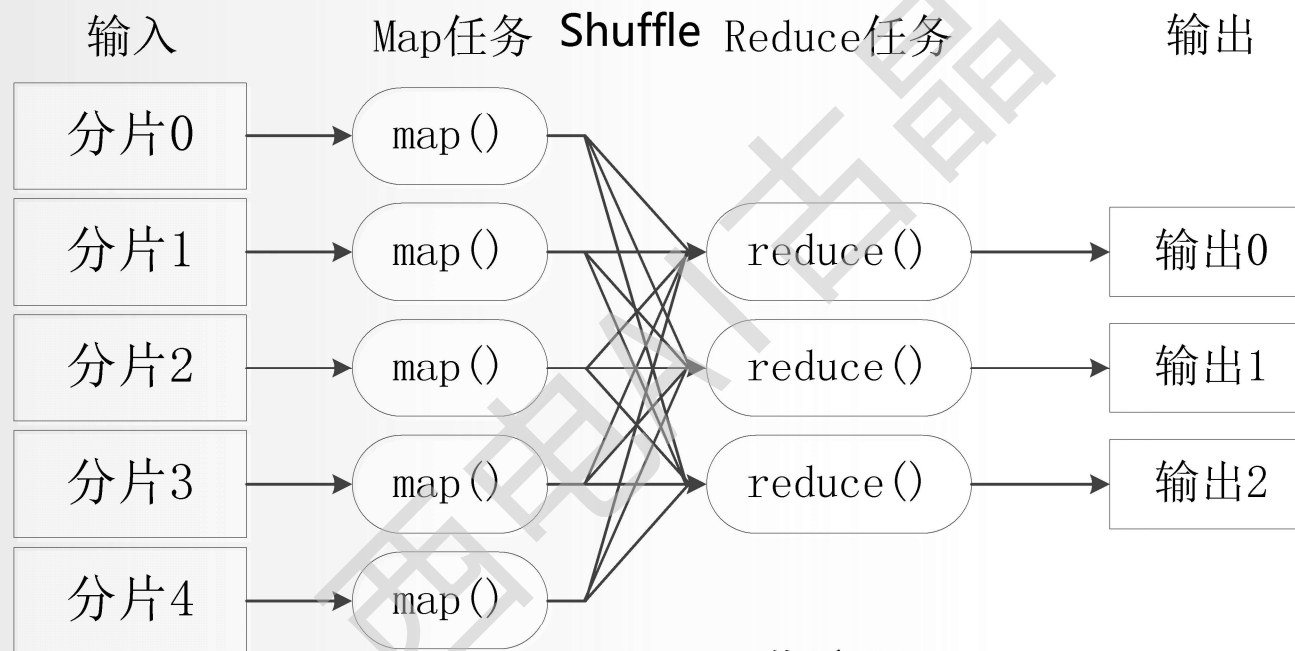
MapReduce设计的一个理念就是“**计算向数据靠拢**”，而不是“**数据向计算靠拢**”，因为移动数据需要大量的网络传输开销，尤其是在大规模数据环境下，这种开销尤为惊人，所以，移动计算要比移动数据更加经济。

本着这个理念，在一个集群中，只要有可能，MapReduce框架就会将Map程序就近地在HDFS数据所在的节点运行，**即将计算节点和存储节点放在一起运行**，从而减少了节点间的数据移动开销。



13.4.1 分布式计算框架MapReduce

(2) MapReduce工作流程



MapReduce工作流程

- 不同的**Map任务**之间不会进行通信。
- 不同的**Reduce任务**之间也不会发生任何信息交换。
- 用户不能显式地从一台机器向另一台机器发送消息。
- **所有的数据交换**都是通过**MapReduce框架自身**去实现的。



13.4.1 分布式计算框架MapReduce



(3) MapReduce的不足之处

总体而言，Hadoop的MapReduce存在以下缺点：

- ① **表达能力有限**。计算都必须转化成Map和Reduce两个操作，但这并不适合所有的情况，难以描述复杂的数据处理过程。
- ② **磁盘IO开销大**。每次执行时都需要从磁盘读取数据，并且在计算完成后需要将中间结果写入到磁盘中，IO开销较大。
- ③ **延迟高**。一次计算可能需要分解成一系列按顺序执行的MapReduce任务，任务之间的衔接由于涉及到IO开销，会产生较高延迟。而且，在前一个任务执行完成之前，其他任务无法开始，因此**难以胜任复杂、多阶段的计算任务**。

由于MapReduce是**基于磁盘**的分布式计算框架，因此，性能方面要逊色于**基于内存**的分布式计算框（比如Spark和Flink等），所以，随着Spark和Flink的发展，MapReduce的市场空间逐渐被挤压，地位也逐渐被边缘化，但是，不可否认的是，MapReduce的一些优秀的设计思想还是在其他框架中得到了很好的继承。



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.2 数据仓库Hive



(1) Hive简介

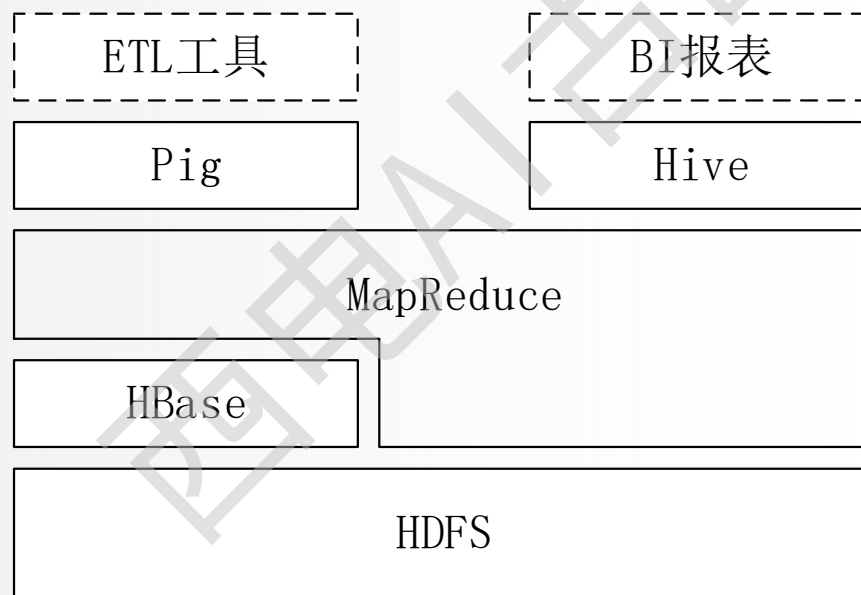
- Hive是一个**构建于Hadoop顶层的数据仓库工具**。
- 某种程度上可以看作是用户编程接口，本身不存储和处理数据
- 依赖分布式文件系统HDFS存储数据。
- 依赖分布式并行计算模型**MapReduce**处理数据。
- 定义了简单的类SQL 查询语言——HiveQL 。
- 用户可以通过编写的HiveQL语句运行MapReduce任务。
- 是一个可以提供有效、合理、直观组织和使用数据的模型。



13.4.2 数据仓库Hive

(2) Hive与Hadoop生态系统中其他组件的关系

Hadoop生态系统



Hadoop生态系统中Hive与其他部分的关系



13.4.2 数据仓库Hive



➤ Hive依赖于HDFS 存储数据

HDFS作为高可靠性的底层存储，用来存储海量数据。

➤ Hive依赖于MapReduce 处理数据

MapReduce对这些海量数据进行处理，实现高性能计算，用HiveQL语句编写的处理逻辑最终均要转化为MapReduce任务来运行。

➤ Pig可以作为Hive的替代工具

pig是一种数据流语言和运行环境，适合用于Hadoop和MapReduce平台上查询半结构化数据集。常用于ETL过程的一部分，即将外部数据装载到Hadoop集群中，然后转换为用户期待的数据格式。

➤ HBase 提供数据的实时访问

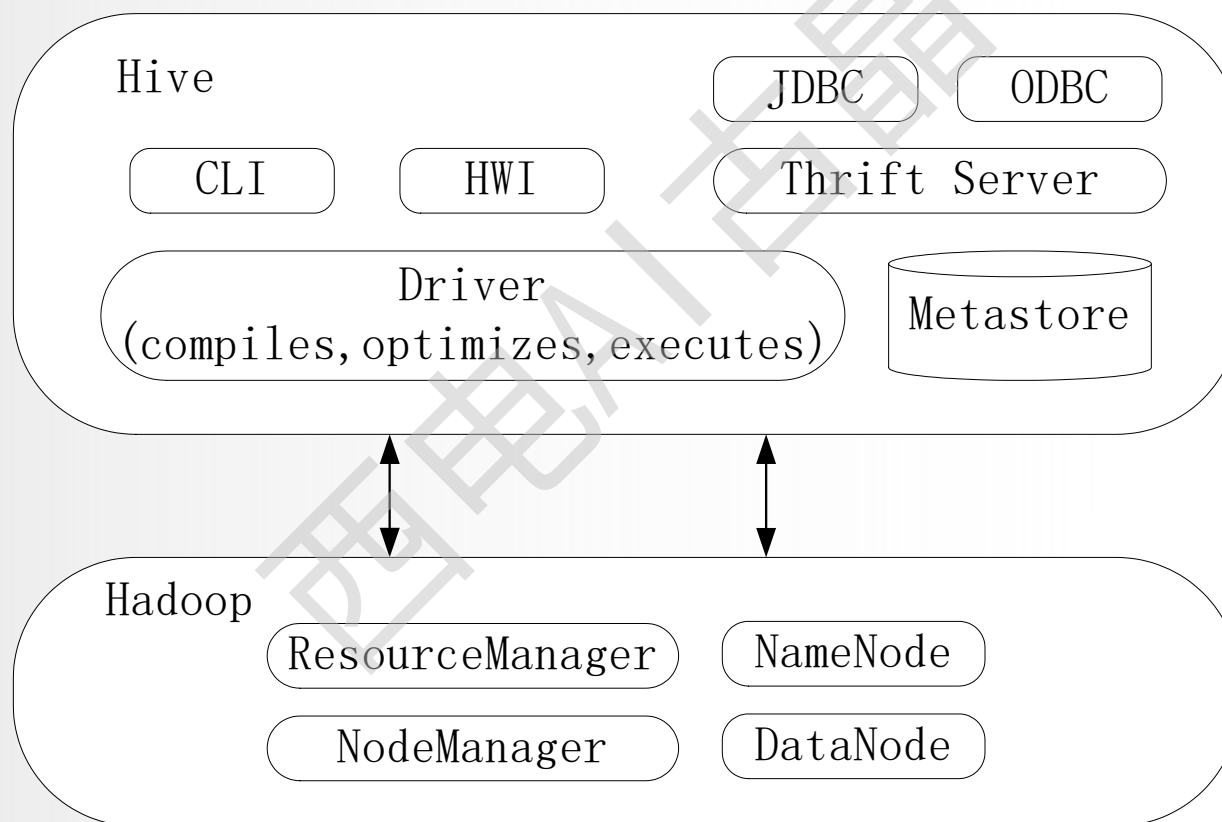
HBase一个面向列的、分布式的、可伸缩的数据库，它可以提供数据的实时访问功能，而Hive只能处理静态数据，主要是BI报表数据，所以HBase与Hive的功能是互补的，它实现了Hive不能提供功能。



13.4.2 数据仓库Hive



(3) Hive系统架构



Hive系统架构



13.4.2 数据仓库Hive



- **用户接口模块**：包括CLI、HWI、JDBC、ODBC、Thrift Server等

CLI是Hive自带的一个命令行界面；

HWI是Hive的一个简单网页界面；

JDBC、ODBC以及Thrift Server可以向用户提供进行编程访问的接口。

- **驱动模块**：包括编译器、优化器、执行器等。

所有命令和查询都会进入到驱动模块，通过该模块对输入进行解析编译，对需求的计算进行优化，然后按照指定的步骤进行执行。

- **元数据存储模块 (Metastore)**：是一个**独立的关系型数据库**。

通常是与MySQL数据库连接后创建的一个MySQL实例，也可以是Hive自带的derby数据库实例。元数据存储模块中主要保存表模式和其他系统元数据，如表的名称、表的列及其属性、表的分区及其属性、表的属性、表中数据所在位置信息等。



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.3 数据仓库Impala

- Impala是由Cloudera公司开发的新型查询系统，它**提供SQL语义**，能查询存储在Hadoop的HDFS和HBase上的PB级大数据。Impala最开始是参照 Dremel系统进行设计的，Impala的目的不在于替换现有的MapReduce工具，而是**提供一个统一的平台用于实时查询**。

ODBC Driver	
Impala	Metastore (Hive)
HDFS	HBase

Impala与其他组件关系



13.4.3 数据仓库Impala



- 与Hive类似，Impala也可以直接与HDFS和HBase进行交互。
- Hive底层执行使用的是MapReduce，所以主要用于处理长时间运行的批处理任务，例如批量提取、转化、加载类型的任务。
- Impala通过与商用并行关系数据库中类似的分布式查询引擎，可以直接从HDFS或者HBase中用SQL语句查询数据，从而大大降低了延迟，主要用于实时查询。
- Impala和Hive采用相同的SQL语法、ODBC 驱动程序和用户接口。



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.4 基于内存的分布式计算框架Spark



(1) Spark简介

- Spark最初由美国加州大学伯克利分校（UC Berkeley）的AMP实验室于2009年开发，是**基于内存计算的大数据并行计算框架**，可用于构建**大型的、低延迟的数据分析应用程序**。
- 2013年Spark加入Apache孵化器项目后发展迅猛，如今已成为Apache软件基金会最重要的三大分布式计算系统开源项目之一（Hadoop、Spark、Storm）。
- Spark在2014年打破了Hadoop保持的基准排序纪录。
 - Spark/206个节点/23分钟/100TB数据。
 - Hadoop/2000个节点/72分钟/100TB数据。
 - Spark用十分之一的计算资源，获得了比Hadoop快3倍的速度。



13.4.4 基于内存的分布式计算框架Spark

Spark具有如下几个主要特点：

- **运行速度快**：使用DAG执行引擎以支持循环数据流与内存计算。
- **容易使用**：支持使用**Scala**、**Java**、**Python**和**R**语言进行编程，可以通过Spark Shell进行交互式编程。
- **通用性**：Spark提供了完整而强大的技术栈，包括**SQL查询**、**流式计算**、**机器学习**和**图算法**组件。
- **运行模式多样**：可运行于独立的集群模式中，可运行于Hadoop中，也可运行于Amazon EC2等云环境中，并且可以访问HDFS、Cassandra、HBase、Hive等多种数据源。



13.4.4 基于内存的分布式计算框架Spark

(2) Spark相对于MapReduce的优点

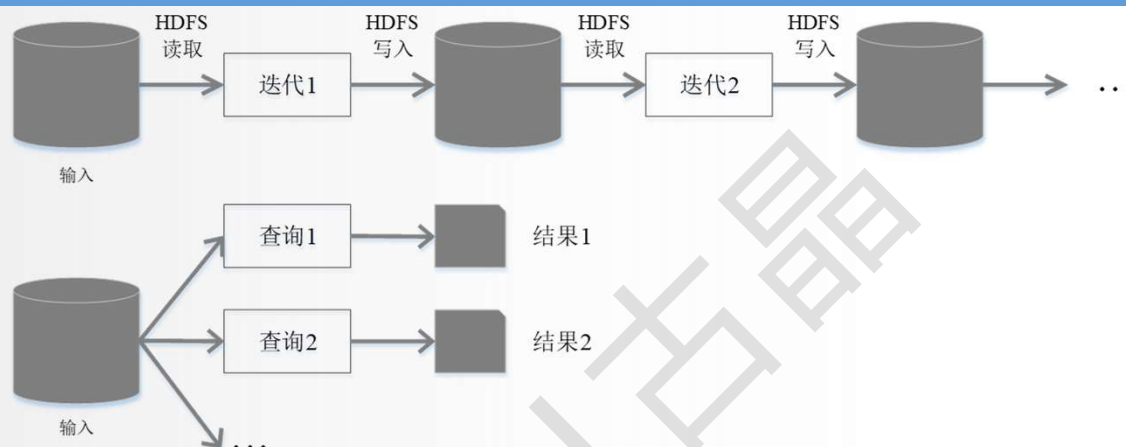
Spark在借鉴Hadoop MapReduce优点的同时，很好地解决了MapReduce所面临的问题。

相比于Hadoop MapReduce，Spark主要具有如下优点：

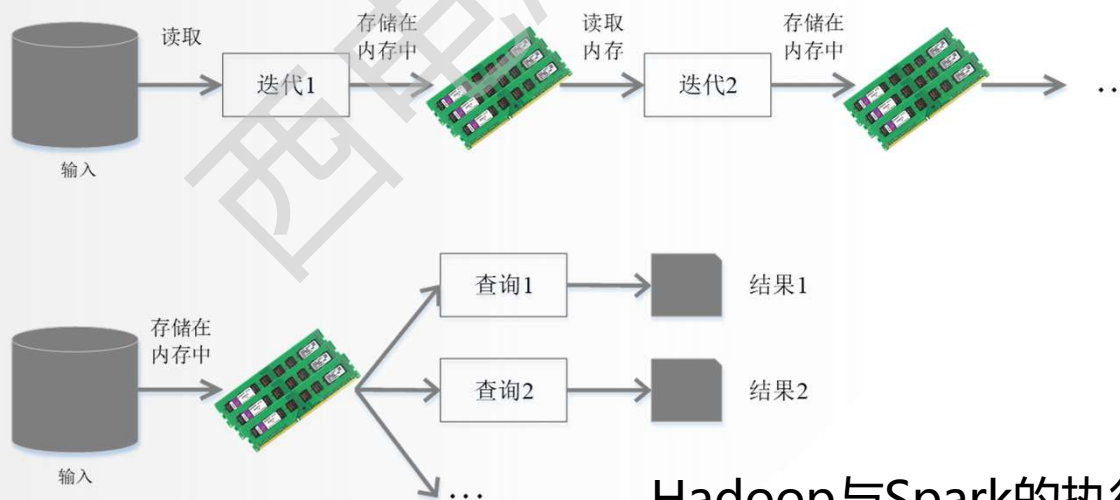
- Spark的**计算模式**也属于MapReduce，但不局限于Map和Reduce操作，还提供了多种数据集操作类型，编程模型比Hadoop MapReduce更灵活。
- Spark提供了**内存计算**，可将中间结果放到内存中，对于迭代运算效率更高。
- Spark**基于DAG的任务调度执行机制**，要优于Hadoop MapReduce的迭代执行机制。



13.4.4 基于内存的分布式计算框架Spark



(a) Hadoop MapReduce 执行流程



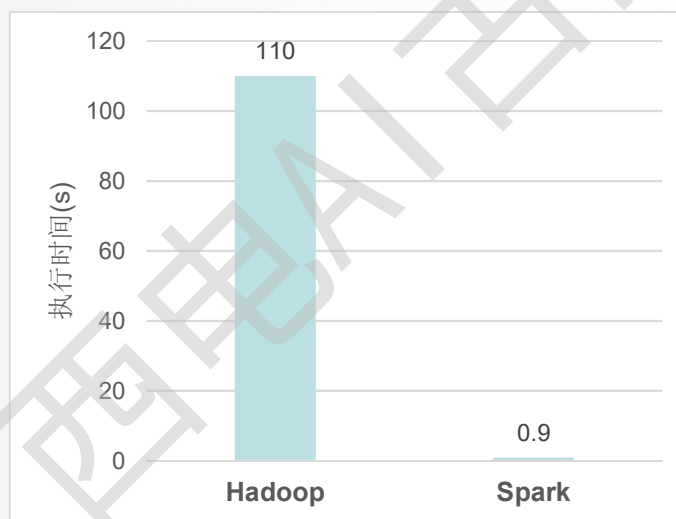
Hadoop与Spark的执行流程对比

(b) Spark 执行流程



13.4.4 基于内存的分布式计算框架Spark

- 使用Hadoop进行**迭代计算**非常耗资源。
- Spark将数据载入内存后，之后的迭代计算都可以直接使用内存中的中间结果作运算，避免了从磁盘中频繁读取数据。



Hadoop与Spark执行逻辑回归的时间对比



13.4.4 基于内存的分布式计算框架Spark



(3) Spark与Hadoop的关系

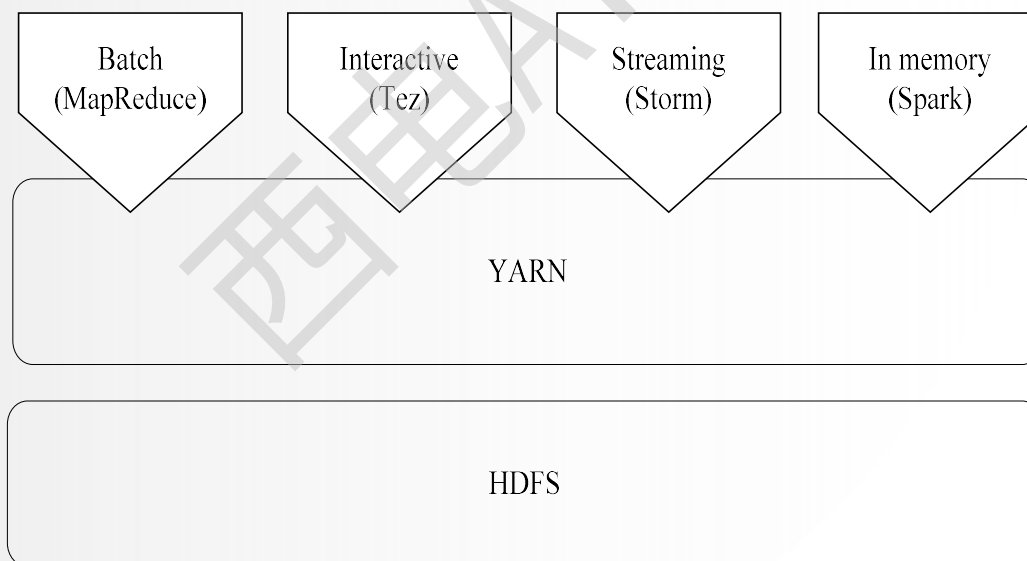
- Spark正以其结构一体化、功能多元化的优势，逐渐成为当今大数据领域最热门的大数据计算平台。目前，越来越多的企业放弃MapReduce，转而使用Spark开发企业应用。
- 但是，Spark作为计算框架，只能解决数据计算问题，无法解决数据存储问题，Spark只取代了Hadoop生态系统中的计算框架MapReduce，而**Hadoop其他组件依然在企业大数据系统中发挥着重要的作用**。
- 比如，企业在采用Spark解决数据计算问题的同时，依然需要依赖Hadoop分布式文件系统HDFS和分布式数据库HBase，来实现不同类型数据的存储和管理，并借助于YARN实现集群资源的管理和调度。
- 因此，**在许多企业实际应用中，Hadoop和Spark的统一部署是一种比较现实合理的选择。**



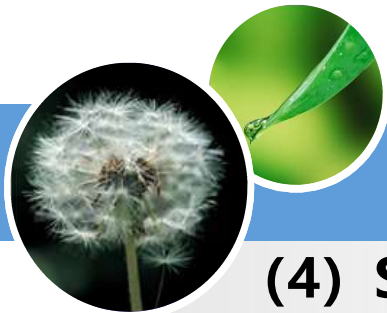
13.4.4 基于内存的分布式计算框架Spark

由于MapReduce、Storm和Spark等，都可以运行在资源管理框架YARN之上，因此，可以在**在YARN之上统一部署各个计算框架**（如图所示）。这些不同的计算框架统一运行在YARN中，可以带来如下好处：

- 计算资源按需伸缩；
- 不用负载应用混搭，集群利用率高；
- 共享底层存储，避免数据跨集群迁移。



Hadoop和Spark的统一部署
西安电子科技大学



13.4.4 基于内存的分布式计算框架Spark

(4) Spark生态系统

在实际应用中，大数据处理主要包括以下三个类型：

- 复杂的**批量数据处理**：通常时间跨度在数十分钟到数小时之间。
- **基于历史数据的交互式查询**：通常时间跨度在数十秒到数分钟之间。
- **基于实时数据流的数据处理**：通常时间跨度在数百毫秒到数秒之间。

当同时存在以上三种场景时，就需要同时部署三种不同的软件，比如 MapReduce / Impala / Storm。这样做难免会带来一些**问题**：

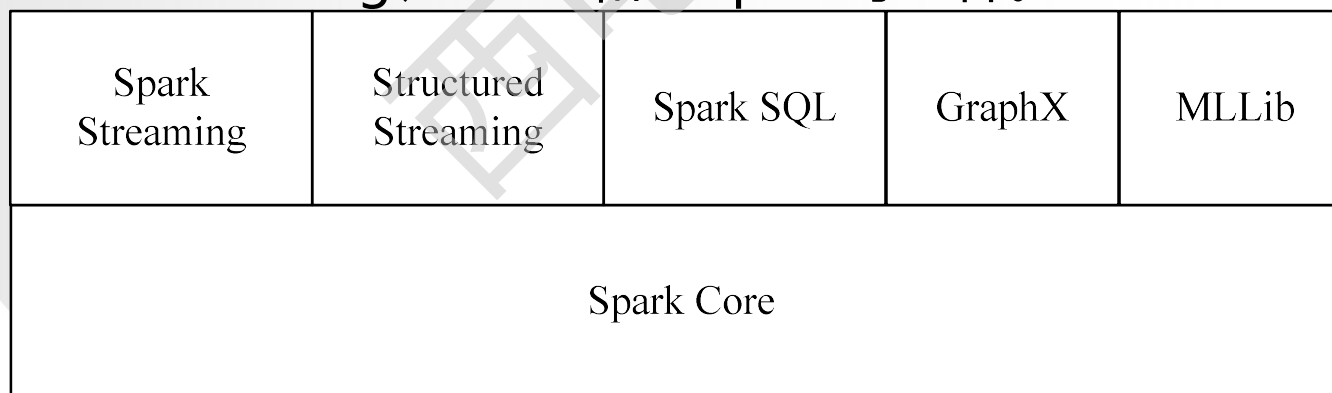
- **不同场景之间输入输出数据无法做到无缝共享**，通常需要进行数据格式的转换。
- **不同的软件需要不同的开发和维护团队**，带来了较高的使用成本。
- **比较难以对同一个集群中的各个系统进行统一的资源协调和分配**。



13.4.4 基于内存的分布式计算框架Spark

- Spark的设计遵循“**一个软件栈满足不同应用场景**”的理念，逐渐形成了一套完整的生态系统。
- 既能够提供**内存计算框架**，也可以支持**SQL查询**、**实时流式计算**、**机器学习和图计算**等。
- Spark可以部署在资源管理器**YARN**之上，提供一站式的大数据解决方案
- 因此，Spark所提供的生态系统足以应对上述三种场景，即同时**支持批处理、交互式查询和流数据处理**。

Spark的生态系统主要包含了Spark Core、Spark SQL、Spark Streaming、Structured Streaming、MLlib和GraphX 等组件。



Spark生态系统
西安电子科技大学



13.4.4 基于内存的分布式计算框架Spark



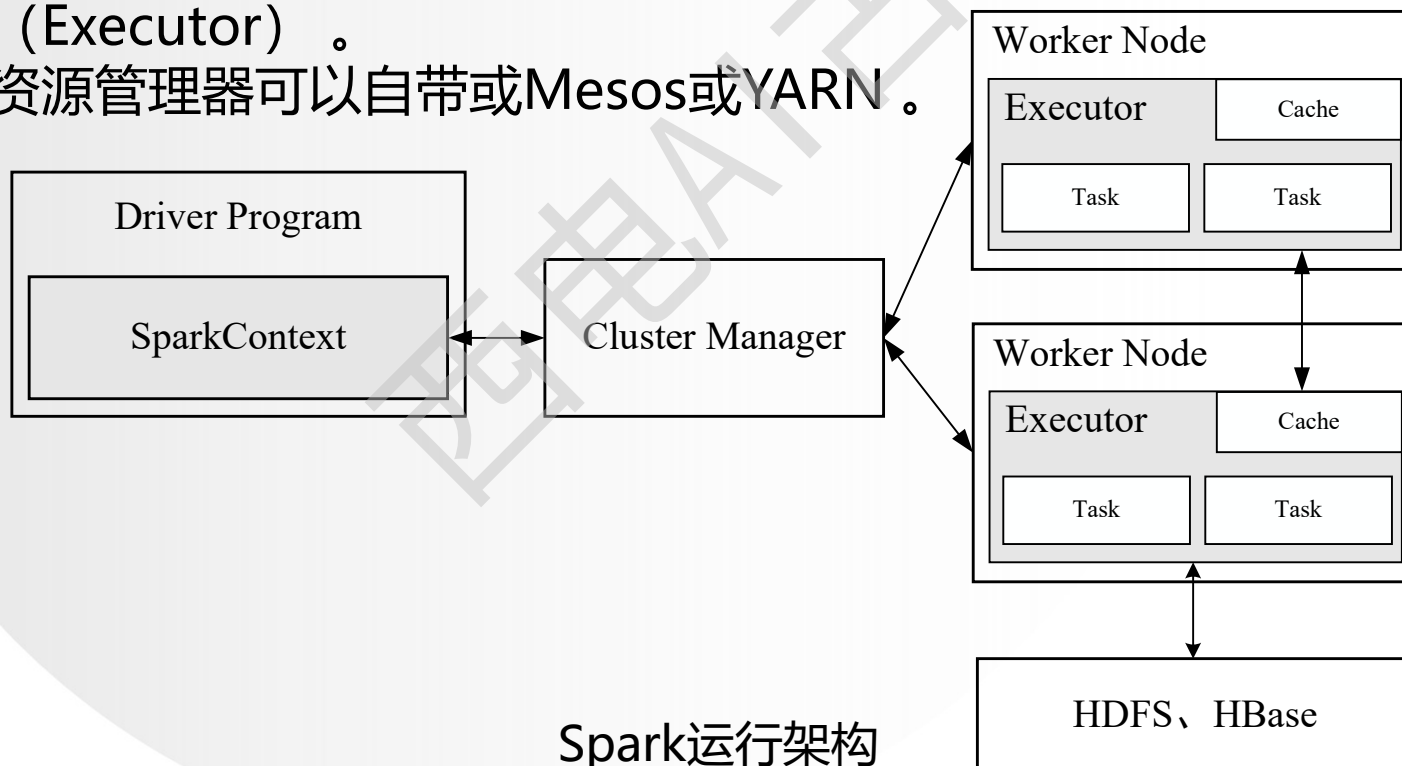
Spark生态系统组件的应用场景

应用场景	时间跨度	其他框架	Spark生态系统中的组件
复杂的批量数据处理	小时级	MapReduce、Hive	Spark
基于历史数据的交互式查询	分钟级、秒级	Impala、Dremel、Drill	Spark SQL
基于实时数据流的数据处理	毫秒、秒级	Storm、S4	Spark Streaming Structured Streaming
基于历史数据的数据挖掘	-	Mahout	MLlib
图结构数据的处理	-	Pregel、Hama	GraphX

13.4.4 基于内存的分布式计算框架Spark

(5) Spark体系架构

- **Spark运行架构**包括**集群资源管理器** (Cluster Manager)、**运行作业任务的工作节点** (Worker Node)、每个应用的**任务控制节点** (Driver) 和每个工作节点上负责具体任务的**执行进程** (Executor)。
- 资源管理器可以自带或Mesos或YARN。





13.4.4 基于内存的分布式计算框架Spark



(6) Spark的数据抽象RDD

- Spark Core是建立在统一的抽象RDD之上，使得Spark的**各个组件可以无缝进行集成，在同一个应用程序中完成大数据计算任务。**
- 一个RDD就是一个**分布式对象集合**，本质上是一个**只读的分区记录集合**，每个RDD可以分成多个分区，每个分区就是一个数据集片段，并且一个RDD的不同分区可以被保存到集群中不同的节点上，从而可以在集群中的不同节点上进行并行计算。



13.4.4 基于内存的分布式计算框架Spark



RDD提供了一组丰富的操作以支持常见的数据运算，分为“**动作**” (Action) 和“**转换**” (Transformation) 两种类型。

- RDD提供的转换接口都非常简单，都是类似map、filter、groupBy、join等**粗粒度的数据转换操作**，而不是针对某个数据项的细粒度修改（不适合网页爬虫）。
- 表面上RDD的功能很受限、不够强大，实际上RDD已经被实践证明可以高效地表达许多框架的编程模型（比如MapReduce、SQL、Pregel）。
- Spark提供了RDD的API，程序员可以通过调用API实现对RDD的各种操作。

RDD典型的执行过程如下：

- RDD读入外部数据源进行创建
- RDD经过一系列的转换（Transformation）操作，每一次都会产生不同的RDD，供给下一个转换操作使用。
- 最后一个RDD经过“动作”操作进行转换，并输出到外部数据源。



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.5 TensorFlowOnSpark

- TensorFlow是一个**开源的、基于Python的机器学习框架**，它是由谷歌公司开发的，并在图形分类、音频处理、推荐系统和自然语言处理等场景下有着丰富的应用，是目前最热门的机器学习框架。TensorFlow是一个采用**数据流图** (Data Flow Graph)、用于数值计算的开源软件库。数据流图中的**节点** (Nodes) 表示**数学操作**，**线**则表示节点间的相互联系的**多维数据组**，即张量(Tensor)。
- 在计算过程中，**张量从图的一端流动到另一端**，这也是这个工具取名为“TensorFlow”的原因。一旦输入端的所有张量准备好，节点将被分配到各种计算设备完成异步并行地执行运算。利用TensorFlow我们可以在多种平台上展开数据分析与计算，如CPU(或GPU)、台式机、服务器、甚至移动设备等等。



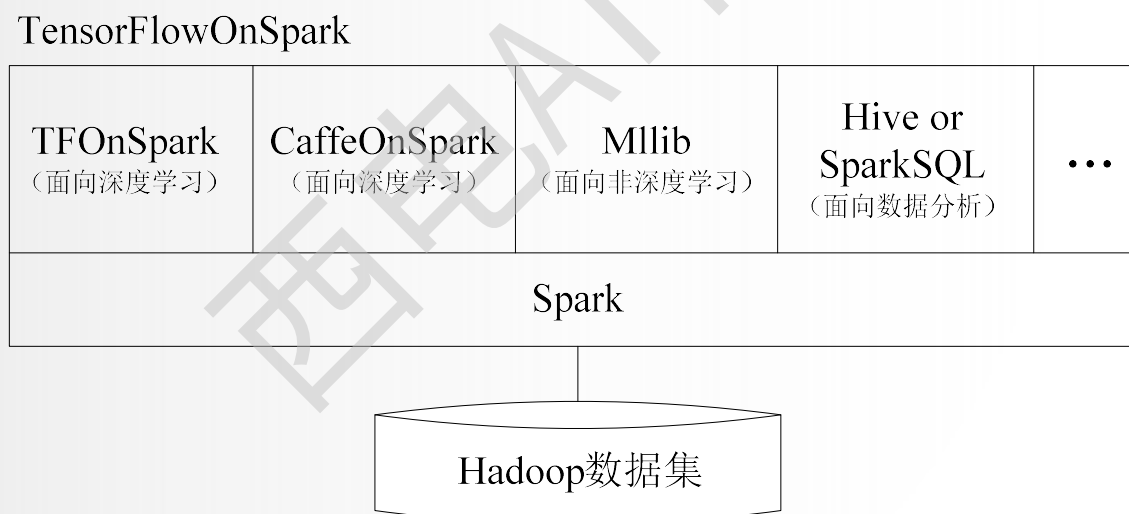
13.4.5 TensorFlowOnSpark

- 尽管TensorFlow也开放了自己的分布式运行框架，但在目前公司的技术架构和使用环境上不是那么友好，如何将TensorFlow加入到现有的环境中（Spark/YARN），并为用户提供更加方便易用的环境，成为了目前所要解决的问题。
- TensorFlowOnSpark项目是由Yahoo开源的一个软件包，**能将TensorFlow与Spark结合在一起使用**，使Spark能够利用TensorFlow拥有深度学习和GPU加速计算的能力。传统情况下处理数据需要跨集群（深度学习集群和Hadoop/Spark集群），Yahoo为了解决跨集群传递数据的问题开发了TensorFlowOnSpark项目。TensorFlowOnSpark目前被用于雅虎私有云中的Hadoop集群，**主要进行大规模分布式深度学习**。



13.4.5 TensorFlowOnSpark

TensorFlowOnSpark在设计时充分考虑了**Spark本身的特性**和**TensorFlow的运行机制**，大大保证了两者的兼容性，使得可以通过较少的修改来运行已经存在的TensorFlow程序。在独立的TFOnSpark程序中能够与SparkSQL、MLlib和其他Spark库一起工作处理数据（如图所示）。



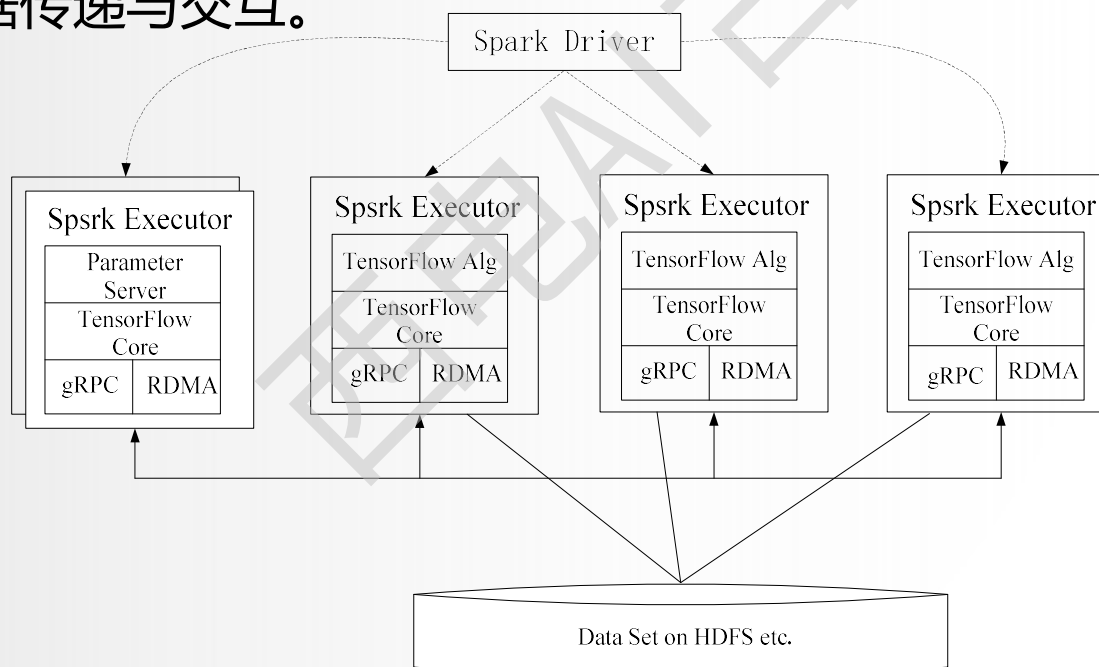
TensorFlowOnSpark与Spark的集成

西安电子科技大学



13.4.5 TensorFlowOnSpark

TensorFlowOnSpark的**体系架构**较为简单（如图所示），Spark Driver程序并不会参与TensorFlow内部相关的计算和处理。其设计思路像是**将一个TensorFlow集群运行在了Spark上**，它会在每个Spark Executor中启动TensorFlow应用程序，然后通过gRPC或RDMA方式进行数据传递与交互。



TensorFlowOnSpark体系架构
西安电子科技大学



13.4.5 TensorFlowOnSpark

TensorFlowOnSpark的Spark应用程序包括4个基本过程：

- (1) **预留**：组建TensorFlow集群，并在每个Executor进程上预留监听端口，启动“数据/控制”消息的监听程序；
- (2) **启动**：在每个Executor进程上启动TensorFlow应用程序；
- (3) **训练/推理**：在TensorFlow集群上完成模型的训练或推理；
- (4) **关闭**：关闭Executor进程上的TensorFlow应用程序，释放相应的系统资源(消息队列)。



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

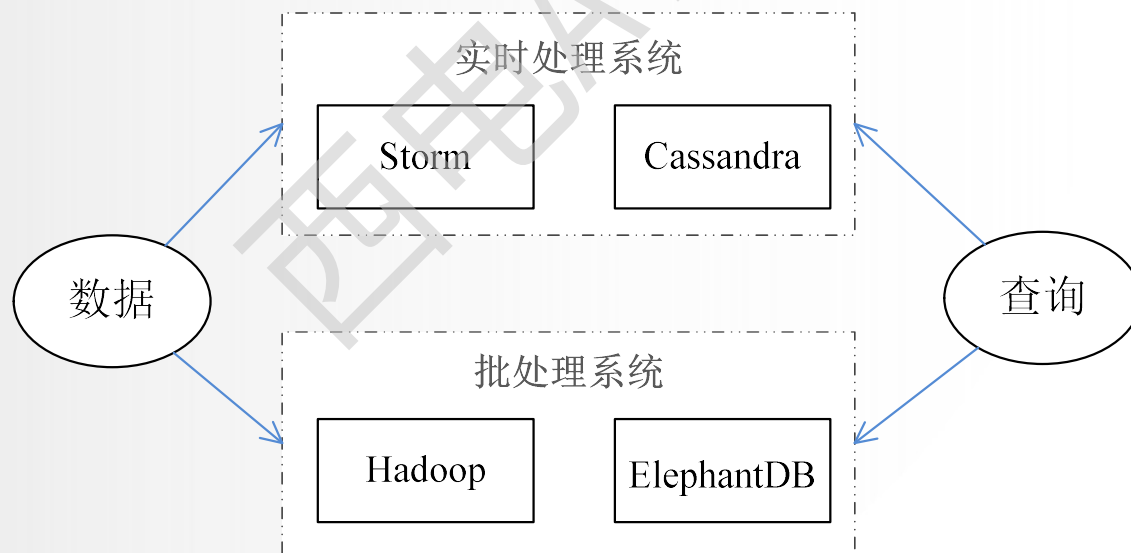
13.4.9 查询分析系统Dremel



13.4.6 流计算框架Storm

(1) Storm简介

- Twitter Storm是一个免费、开源的分布式实时计算系统，Storm对于实时计算的意义类似于Hadoop对于批处理的意义，Storm可以简单、高效、可靠地处理流数据，并支持多种编程语言。
- Storm框架可以方便地与数据库系统进行整合，开发出强大的实时计算系统。
- Twitter是全球访问量最大的社交网站之一，Twitter开发Storm流处理框架也是为了应对其不断增长的流数据实时处理需求。



Twitter的分层数据处理架构
西安电子科技大学



13.4.6 流计算框架Storm

(2) Storm的特点

- Storm可用于许多领域中，如实时分析、在线机器学习、持续计算、远程RPC、数据提取加载转换等。
- Storm具有以下主要特点：
 - **整合性**：Storm可方便地与队列系统和数据库系统进行整合。
 - **简易的API**：Storm的API在使用上即简单又方便。
 - **可扩展性**：Storm的并行特性使其可以运行在分布式集群中。
 - **容错性**：Storm可自动进行故障节点的重启、任务的重新分配。
 - **可靠的消息处理**：Storm保证每个消息都能完整处理。
 - **支持各种编程语言**：Storm支持使用各种编程语言来定义任务。
 - **快速部署**：Storm可以快速进行部署和使用。
 - **免费、开源**：Storm是一款开源框架，可以免费使用。



13.4.6 流计算框架Storm

(3) Storm的框架设计

- Storm 运行任务的方式与Hadoop类似：Hadoop运行的是MapReduce作业，而Storm运行的是“Topology”。
- 但两者的任务大不相同，主要的不同是：MapReduce作业最终会完成计算并结束运行，而**Topology将持续处理消息**（直到人为终止）。

Storm和Hadoop架构组件功能对应关系

	Hadoop	Storm
应用名称	Job	Topology
系统角色	JobTracker	Nimbus
	TaskTracker	Supervisor
组件接口	Map/Reduce	Spout/Bolt



13.4.6 流计算框架Storm

Storm集群采用 “**Master—Worker**” 的节点方式:

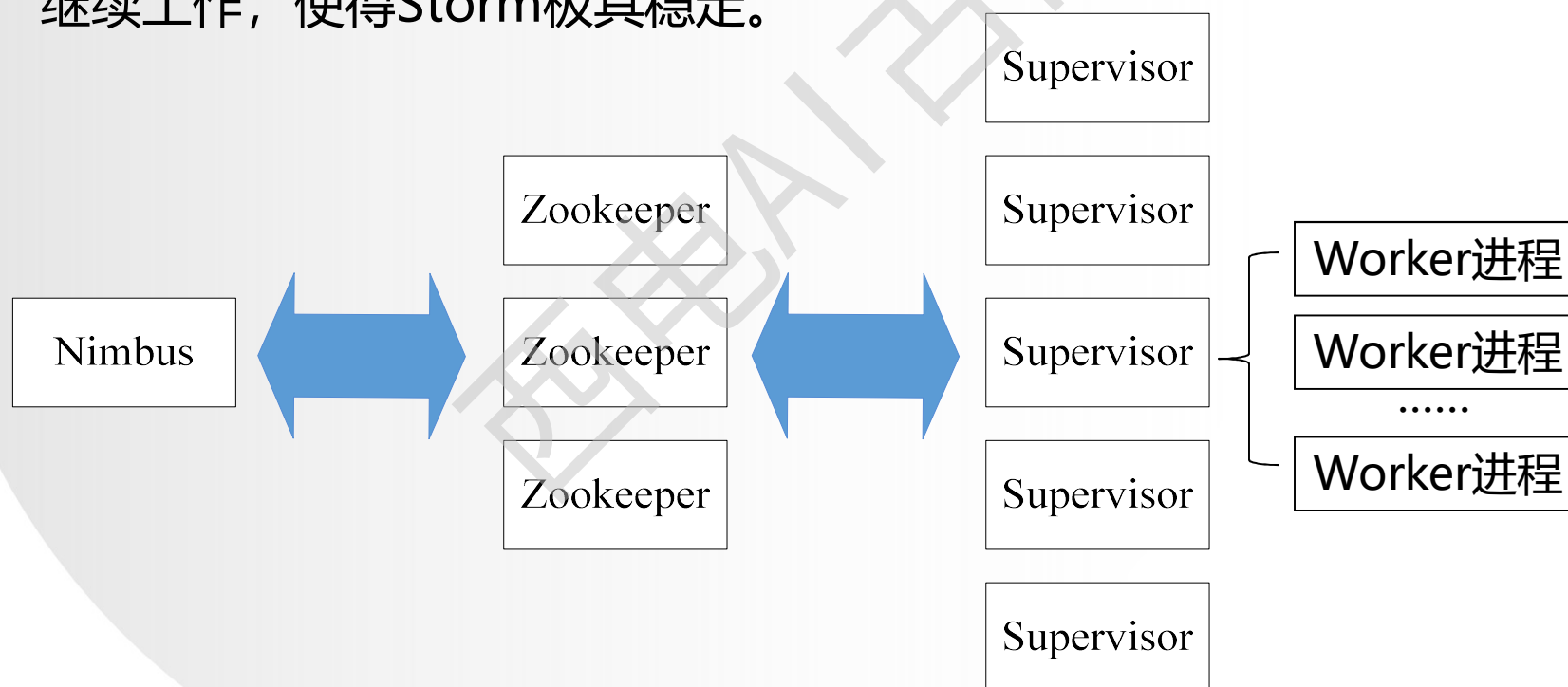
- Master节点运行名为 “Nimbus” 的后台程序（类似Hadoop中的 “JobTracker” ），**负责在集群范围内分发代码、为Worker分配任务和监测故障。**
- Worker节点运行名为 “Supervisor” 的后台程序，**负责监听分配给它所在机器的工作**，即根据Nimbus分配的任务来决定启动或停止Worker进程，一个Worker节点上同时运行若干个Worker进程。



13.4.6 流计算框架Storm



- Storm使用Zookeeper来作为**分布式协调组件**，负责Nimbus和多个Supervisor之间的所有协调工作。借助于Zookeeper，若Nimbus进程或Supervisor进程意外终止，重启时也能读取、恢复之前的状态并继续工作，使得Storm极其稳定。



Storm集群架构示意图
西安电子科技大学

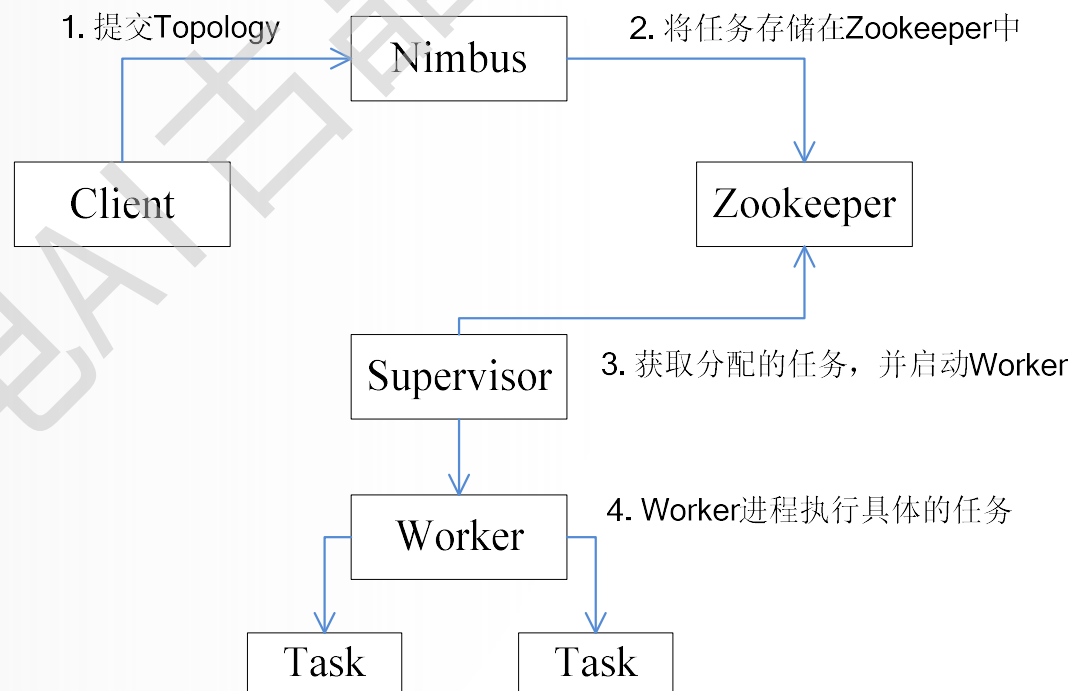


13.4.6 流计算框架Storm



- **所有Topology任务的提交必须在Storm客户端节点上进行**，提交后，由Nimbus节点**分配**给其他Supervisor节点进行处理。
- Nimbus节点首先将提交的Topology进行**分片**，分成一个个Task，**分配**给相应的Supervisor，并将Task和Supervisor相关的信息**提交**到Zookeeper集群上。
- Supervisor会去Zookeeper集群上**认领**自己的Task，**通知**自己的Worker进程进行Task的处理。
- **说明：**在提交了一个Topology之后，Storm就会创建Spout/Bolt实例并进行序列化。之后，将序列化的组件发送给所有的任务所在的机器(即Supervisor节点)，在每一个任务上反序列化组件。

Storm的**工作流程**如下图所示：



Storm工作流程示意图



13.4.6 流计算框架Storm

(4) Spark Streaming与Storm的对比

- Spark Streaming和Storm最大的**区别**在于，Spark Streaming无法实现毫秒级的流计算，而**Storm可以实现毫秒级响应**。
- Spark Streaming**构建在Spark上**，一方面是因为Spark的低延迟执行引擎（100ms+）可以用于实时计算，另一方面，相比于Storm，RDD数据集更容易做高效的容错处理。
- Spark Streaming采用的小批量处理的方式使得它可以同时**兼容批量和实时数据处理的逻辑和算法**，因此，方便了一些需要历史数据和实时数据联合分析的特定应用场合。



13.4.6 流计算框架Storm

- 从编程的灵活性来讲，Storm是比较理想的选择，它使用Apache Thrift，**可以用任何编程语言来编写拓扑结构** (Topology)。
- 当需要在一个集群中把流计算和图计算、机器学习、SQL查询分析等进行结合时，可以选择Spark Streaming，因为，在Spark上可以**统一部署** Spark SQL，Spark Streaming、MLlib，GraphX等组件，提供便捷的一体化编程模型。
- 当应用场景需要**毫秒级响应**时，可以选择Storm，因为Spark Streaming无法实现毫秒级的流计算。



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.7 流计算框架Flink

(1) Flink简介

- Flink是Apache软件基金会的顶级项目之一，是一个针对流数据和批数据的分布式计算框架，设计思想主要来源于Hadoop、MPP数据库、流计算系统等。
- Flink主要是由**Java代码实现**的，目前主要还是依靠开源社区的贡献而发展。
- Flink所要处理的**主要场景是流数据**，批数据只是流数据的一个特例而已，也就是说，Flink会把所有任务当成流来处理。Flink可以支持本地的快速迭代以及一些环形的迭代任务。



13.4.7 流计算框架Flink

Flink**以层级式系统形式组建其软件栈**（如图所示），不同层的栈建立在其下层基础上。具体而言，Flink的典型特性如下：

- 提供了**面向流处理**的DataStream API和**面向批处理**的DataSet API。DataSet API 支持Java、Scala和Python，DataStream API 支持Java和Scala；
- 提供了**多种候选部署方案**，比如本地模式（Local）、集群模式（Cluster）和云模式（Cloud）。对于集群模式而言，可以采用独立模式（Standalone）或者YARN；
- 提供了一些**类库**，包括Table（处理逻辑表查询）、FlinkML（机器学习）、Gelly（图像处理）和CEP（复杂事件处理）；
- 提供了较好的**Hadoop兼容性**，不仅可以支持YARN，还可以支持HDFS、HBase等数据源。



13.4.7 流计算框架Flink

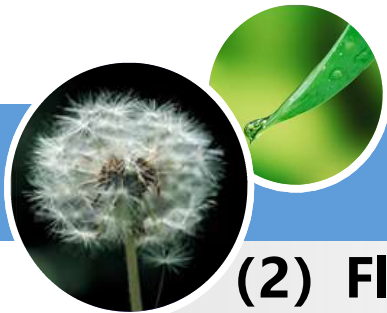
Deploy	Core	APIs & Libraries					
		CEP Event Processing				FlinkML Machine Learning	
		Table Relational				Gelly Graph Processing	
		Table Relational				Table Relational	
		DataStream API Stream Processing			DataSet API Batch Processing		
		Runtime Distributed Streaming Dataflow					
		Local Single JVM		Cluster Standalone , YARN		Cloud GCE , EC2	

Flink架构图



13.4.7 流计算框架Flink

- Flink和Spark一样，都是基于内存的计算框架，因此，都可以获得较好的实时计算性能。
- 当全部运行在Hadoop YARN之上时，Flink的性能甚至还要略好于Spark，因为，Flink支持增量迭代，具有对迭代进行自动优化的功能。
- Flink和Spark Streaming都**支持流计算**，二者的区别在于Flink是一行一行地处理数据，而Spark Streaming是基于RDD的小批量处理，所以Spark Streaming在流式处理方面不可避免地会增加一些延时，**实时性**没有Flink好。
- Flink的流计算性能和Storm差不多，可以支持毫秒级的响应，而Spark Streaming则只能支持秒级响应。
- 总体而言，Flink和Spark都是非常优秀的基于内存的分布式计算框架，但是，**Spark的市场影响力和社区活跃度明显超过Flink，这在一定程度上限制了Flink的发展空间。**



13.4.7 流计算框架Flink

(2) Flink是理想的流计算框架

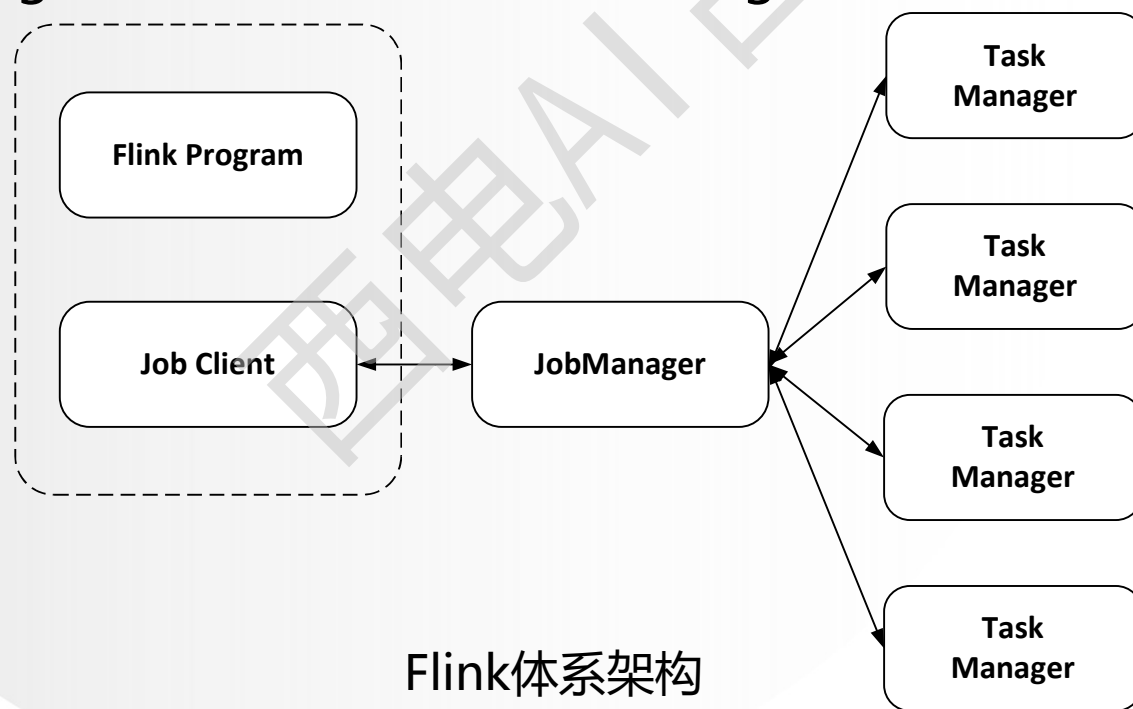
- 流处理架构需要具备**低延迟、高吞吐和高性能**的特性，而目前从市场上已有的产品来看，只有Flink可以满足要求。
- **Storm**虽然可以做到低延迟，但是无法实现高吞吐，也不能在故障发生时准确地处理计算状态。
- **Spark Streaming**通过采用微批处理方法实现了高吞吐和容错性，但是牺牲了低延迟和实时处理能力。
- **Structured Streaming**采用持续处理模型时可以支持毫秒级的实时响应，但是，这是以牺牲一致性为代价的，持续处理模型只能做到“至少一次”的一致性，而无法保证端到端的完全一致性。
- Flink实现了**Google Dataflow**流计算模型，是一种兼具**高吞吐、低延迟和高性能**的实时流计算框架，并且同时支持**批处理和流处理**。此外，Flink支持**高度容错的状态管理**，防止状态在计算过程中因为系统异常而出现丢失。因此，**Flink就成为了能够满足流处理架构要求的理想的流计算框架。**



13.4.7 流计算框架Flink

(3) Flink体系架构

如图所示，Flink系统主要由两个组件组成，分别为**JobManager**和**TaskManager**，Flink 架构也遵循Master-Slave架构设计原则，JobManager为Master节点，TaskManager为Slave节点。



Flink体系架构

西安电子科技大学



13.4 大数据处理与分析代表性产品

13.4.1 分布式计算框架MapReduce

13.4.2 数据仓库Hive

13.4.3 数据仓库Impala

13.4.4 基于内存的分布式计算框架Spark

13.4.5 TensorFlowOnSpark

13.4.6 流计算框架Storm

13.4.7 流计算框架Flink

13.4.8 大数据编程框架Beam

13.4.9 查询分析系统Dremel



13.4.8 大数据编程框架Beam

在大数据处理领域，开发者经常要用到很多不同的技术、框架、API、开发语言和SDK。

- 根据不同的企业业务系统开发需求，开发者很可能会用MapReduce进行**批处理**，用Spark SQL进行**交互式查询**，用Flink实现**实时流处理**，还有可能用到**基于云端的机器学习框架**。大量的开源大数据产品（比如MapReduce、Spark、Flink、Storm、Apex等）为**大数据开发者提供了丰富的工具的同时，也增加了开发者选择合适工具的难度**，尤其对于新入行的开发者来说更是如此。
- 新的分布式处理框架可能带来更高的性能、更强大的功能和更低的延迟，但是，用户切换到新的分布式处理框架的代价也非常大——**需要学习一个新的大数据处理框架，并重写所有的业务逻辑。**



13.4.8 大数据编程框架Beam



解决这个问题的思路包括两个部分：

- **首先**，需要一个编程范式，能够统一、规范分布式数据处理的需求，**例如**，统一批处理和流处理的需求；
- **其次**，生成的分布式数据处理任务，应该能够在各个分布式执行引擎（如Spark、Flink等）上执行，用户可以自由切换分布式数据处理任务的执行引擎与执行环境。**Apache Beam**的出现，就是为了解决这个问题。



13.4.8 大数据编程框架Beam

- Beam是由谷歌贡献的Apache顶级项目，它的目标是为开发者提供一个易于使用、却又很强大的数据并行处理模型，能够支持**流处理和批处理**，并**兼容多个运行平台**。
- Beam是一个**开源的统一的编程模型**，开发者可以使用Beam SDK来创建数据处理管道，然后，这些程序可以在任何支持的执行引擎上运行，比如运行在Apex、Spark、Flink、Cloud Dataflow上。
- Beam SDK定义了**开发分布式数据处理任务业务逻辑的API接口**，即提供一个统一的编程接口给到上层应用的开发者，开发者不需要了解底层的具体的大数据平台的开发接口是什么，直接通过Beam SDK的接口，就可以开发数据处理的加工流程，不管输入是用于批处理的有限数据集，还是用于流处理的无限数据集。
- 对于有限或无限的输入数据，Beam SDK都使用相同的类来表现，并且使用相同的转换操作进行处理。

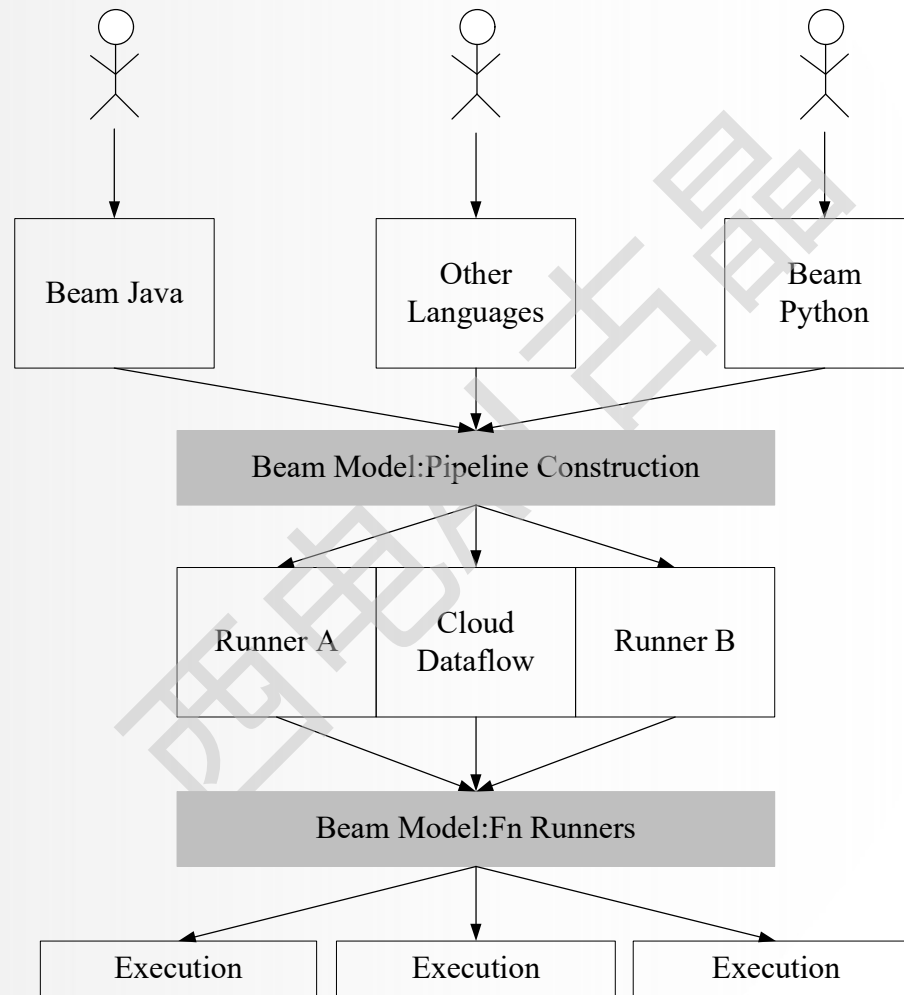


13.4.8 大数据编程框架Beam

- 如图所示，终端用户用Beam来实现自己所需的流计算功能，使用的终端语言可能是Python、Java等，Beam**为每种语言提供了一个对应的SDK**，用户可以使用相应的SDK创建数据处理管道，用户写出的程序可以被运行在各个Runner上，每个Runner都实现了从Beam管道到平台功能的映射。
- 目前主流的大数据处理框架Flink、Spark、Apex以及谷歌的Cloud DataFlow等，都有了支持Beam的Runner。通过这种方式，Beam**使用一套高层抽象的API屏蔽了多种计算引擎的区别**，开发者只需要编写一套代码就可以运行在不同的计算引擎之上（比如Apex、Spark、Flink、Cloud Dataflow等）。



13.4.8 大数据编程框架Beam



Beam使用一套高层抽象的API屏蔽多种计算引擎的区别
西安电子科技大学



13.4 大数据处理与分析代表性产品

- 13.4.1 分布式计算框架MapReduce
- 13.4.2 数据仓库Hive
- 13.4.3 数据仓库Impala
- 13.4.4 基于内存的分布式计算框架Spark
- 13.4.5 TensorFlowOnSpark
- 13.4.6 流计算框架Storm
- 13.4.7 流计算框架Flink
- 13.4.8 大数据编程框架Beam
- 13.4.9 查询分析系统Dremel**



13.4.9 查询分析系统Dremel



- Dremel是一种**可扩展的、交互式的实时查询系统**，用于只读嵌套数据的分析。
- 通过结合多级树状执行过程和列式数据结构，它能做到几秒内完成对万亿张表的聚合查询。系统可以扩展到成千上万的CPU上，满足谷歌公司上万用户操作PB级的数据，可以在2到3秒内完成PB级别数据的查询。

Dremel具有以下几个主要的**特点**：

- (1) Dremel是一个大规模、稳定的系统。
- (2) Dremel是MapReduce交互式查询能力不足的补充。
- (3) Dremel的**数据模型是嵌套**的。
- (4) Dremel中的数据是用**列式存储**的。
- (5) Dremel结合了**Web搜索**和**并行DBMS** (Database Management System) 的技术。



第13章 数据存储与管理



- 13.1 数据处理与分析的概念
- 13.2 机器学习和数据挖掘算法
- 13.3 大数据处理与分析技术
- 13.4 大数据处理与分析代表性产品
- 13.5 知识点小结



13.5 知识点小结

本章知识小结:

- 数据分析与数据挖掘的区别
- 典型的机器学习和数据挖掘算法
- 大数据处理分析技术及其代表性产品



Thanks!

See you